

UNIVERSIDAD SURCOLOMBIANA

Faculty of Engineering
Software Engineering Program
Artificial Intelligence Course — BEINSOF52

CodeSentinel AI

A Local-First, Privacy-Preserving AI Code Reviewer

Project 2 — Artificial Intelligence Applied to Software Engineering

Submitted by

Linda Valentina López Rubiano 20212199950
Juan Felipe Andrade 20212200010

Course professor

Juan Antonio Castro Silva, Ph.D.

Neiva, Huila — Colombia
May 2026

Table of Contents

Abstract 5

1. Introduction 6

 Document Structure 7

2. Problem Statement 7

 2.1 Research Gap 8

 2.2 Research Question..... 8

3. Objectives 8

 3.1 General Objective..... 8

 3.2 Specific Objectives..... 9

4. State of the Art, Background and Related Work 9

 4.1 Foundational Concepts..... 9

 4.1.1 Transformer Language Models 9

 4.1.2 Quantisation 10

 4.1.3 Local Inference Runtimes..... 10

 4.1.4 Static Analysis 10

 4.1.5 LLM-as-a-Judge and Tool Grounding 11

 4.2 Existing Tools..... 11

 4.3 Relationship to This Work..... 12

5. Requirements..... 12

 5.1 Functional Requirements 12

 5.2 Non-Functional Requirements..... 14

6. Use Cases and User Stories..... 15

 6.1 Actors..... 15

 6.2 User Stories 16

 6.3 Use Case Descriptions 16

 UC-01 — Review a File 16

 UC-02 — Review a Pasted Snippet..... 17

 UC-03 — Review a Git Diff 17

 UC-04 — Export a Report 17

 UC-05 — Diagnose the Environment 17

7. Use Case Diagram 18

8. Data Dictionary and Entity–Relationship Model 19

8.1 Entity: Finding 19

8.2 Entity: ReviewResult..... 20

8.3 Entity–Relationship Model 21

9. Class Diagram..... 22

9.1 Responsibilities of Each Class..... 22

9.2 Design Patterns Applied 23

10. Graphical User Interface Design (Mockups) 23

10.1 Header 24

10.2 Input Area 24

10.3 Results Area 25

10.4 Accessibility and Internationalisation 26

11. Web Services Catalogue and API Documentation 27

11.1 Consumed Service — Ollama Inference API 27

Example Request (abbreviated)..... 27

Example Response (abbreviated) 28

11.2 Exposed Service — Web Interface API 28

11.3 Command-Line Interface 29

Example Invocation (Bash) 30

12. Testing (Unit, Functional and Integration) 30

12.1 Unit Tests 30

12.2 Functional Tests 31

12.3 Integration Tests 32

12.4 Code Coverage 32

13. Proposed Architecture 33

13.1 Review Pipeline in Detail 34

13.2 On the Role of the Model 35

14. Results and Discussion 35

14.1 Evaluation Method 36

14.2 Detection Results 36

14.2.1 Python — user_service.py 36

14.2.2 Go — payments.go	37
14.2.3 TypeScript and Java	37
14.2.4 Aggregate Detection Numbers.....	37
14.3 Performance.....	38
14.4 Reproducibility	39
14.5 Discussion	39
14.6 Threats to Validity	40
15. Recommendations and Future Work.....	40
15.1 Cross-File Context.....	41
15.2 Abstract-Syntax-Tree-Aware Chunking	41
15.3 Editor Integration	41
15.4 Pre-Commit and CI Hook Packages	41
15.5 A Larger Model on Capable Hardware	41
15.6 A Published Benchmark	42
15.7 Multi-Turn Repair Loop	42
15.8 Telemetry, Strictly Opt-In	42
16. Conclusion.....	42
References	43
Appendix A — Technology Stack	46
Appendix B — Reproducing the System	48
Source-Code Statistics.....	48

Abstract

CodeSentinel AI is a code-review assistant that runs entirely on the developer’s own computer. It reads a source file, a pasted snippet, or the changes in a Git commit and returns a structured report of defects, security vulnerabilities, performance problems, style violations, and missing documentation. Each finding carries a severity level, a line reference, an explanation of why the issue matters, and a concrete suggested fix.

The system is built around a quantised seven-billion-parameter coding language model — Qwen2.5-Coder — served locally through the Ollama runtime. No source code ever leaves the machine: the only network connection the application opens is to the loopback address 127.0.0.1:11434, where the model runs. Around that model we designed and implemented an original review engine that segments large files, asks the model for a strict JSON verdict, repairs malformed responses, and fuses the model’s judgement with the output of classic static analysers (Bandit, Ruff, ESLint). The same engine drives two interchangeable front ends: a colourised terminal application built with Typer and Rich, and a graphical web/desktop interface built with Gradio and pywebview.

This document describes the problem space, our objectives, the state of the art, the system requirements, the design (use cases, entity–relationship model, class structure, interface mockups and service catalogue), the testing strategy, the proposed architecture, the measured results on a curated benchmark with planted defects, and our recommendations for future work. The system was evaluated on four files across four languages totalling 38 planted defects, of which 32+ were detected, including all critical security vulnerabilities in the Python sample. End-to-end review latency stays under 30 seconds on warm inference, memory use remains under 6 GB, and reproducibility is guaranteed by a fixed random seed.

Keywords: *local large language model, code review, static analysis, Qwen2.5-Coder, Ollama, on-device inference, privacy-preserving artificial intelligence, software engineering.*

1. Introduction

Software teams produce code at a rate that has long outstripped the capacity of human reviewers to keep up with it. Code review remains one of the most effective practices for catching defects before they reach production (Bacchelli & Bird, 2013); yet it is slow, it depends heavily on the reviewer's experience and attention, and it does not scale with the size of a codebase. Over the last two years a class of tools has emerged that applies large language models (LLMs) to this task. Products such as GitHub Copilot's pull request review, Cursor Review, and CodeRabbit can read a diff and flag real problems with a quality comparable to that of a competent engineer.

These tools share one property that limits where they can be used: they are cloud services. Every line of code submitted for review is uploaded to a third-party server. For a student handling coursework this is merely a privacy preference; for a company in a regulated sector such as banking or healthcare it is often a legal impossibility under data-protection regimes (GDPR in Europe, HIPAA in the United States, Habeas Data in Colombia); and for an individual developer it is a recurring subscription cost that grows linearly with team size.

CodeSentinel AI was built to answer a single, focused question: *Can we obtain the same class of review quality while keeping all source code on the developer's own machine?* The project is the second of two assignments for the Artificial Intelligence course at Universidad Surcolombiana. Its theme — “Artificial Intelligence applied to Software Engineering” — is open, and the assignment specifies three constraints: the model must run locally, the application must offer both a graphical user interface and a terminal interface, and the work must be carried out by a team of two.

We chose to build an AI-powered code reviewer. The model that performs the linguistic reasoning is a pre-existing open-source model that runs on-device; the engineering contribution of this project — the part designed and implemented from scratch — is everything that turns that raw model into a usable, reliable reviewing tool: the prompt design, the response parser, the chunking and merging logic, the integration with deterministic static analysers, the reporting layer, and the two front ends. The remainder of this document describes that contribution in detail, following the structure prescribed by the course rubric.

Document Structure

The remainder of this report is organised as follows. Section 2 frames the problem and the gap that motivates the system. Section 3 states the general and specific objectives. Section 4 surveys the state of the art and positions our work against existing tools. Section 5 enumerates the functional and non-functional requirements. Sections 6 and 7 describe the use cases and the corresponding diagram. Section 8 presents the data dictionary and the entity–relationship model. Section 9 contains the class diagram. Section 10 documents the graphical interface design through mockups. Section 11 catalogues the consumed and exposed web services and the command-line interface. Section 12 details the testing strategy. Section 13 describes the proposed architecture. Section 14 reports the experimental results and discusses them. Section 15 lists recommendations and future work, and Section 16 concludes.

2. Problem Statement

Manual code review has three well-documented weaknesses. First, it is slow, because it competes for a senior engineer’s time — time that is also needed for design, planning, mentoring, and the engineer’s own implementation work. Second, it is inconsistent: two reviewers, or the same reviewer on two different days, apply different standards depending on workload, mood, and recent experience. Third, and most importantly, it does not scale: the volume of code that needs review grows with the size of the team, but reviewing capacity does not.

Automated tooling addresses parts of this. Linters and static analysers such as Ruff, Bandit, and ESLint are fast and deterministic, but they are rule-based: they recognise only the patterns their authors encoded, they cannot reason about intent, and they are blind to whole categories of problem — a misleading variable name, a logic error that is syntactically valid, a security vulnerability that depends on how a value flows through the program, or the absence of an await on an asynchronous call.

Cloud-hosted LLM reviewers close that reasoning gap, but at a substantial cost. They transmit proprietary source code to external infrastructure outside the developer’s legal and operational control, they require a paid subscription that scales with usage, and they

depend on network availability — making them unusable offline and unreliable on slow connections.

2.1 Research Gap

The gap we address is precise: **there is no widely available reviewer that combines the semantic reasoning of a language model with the privacy guarantee and zero marginal cost of a tool that runs entirely on the local machine.** CodeSentinel AI occupies exactly that gap.

2.2 Research Question

The work is guided by one central question and three supporting questions.

- **Central** — Can a seven-billion-parameter coding model running on consumer hardware deliver code-review quality comparable to that of cloud reviewers, while preserving the privacy of the reviewed source code?
- **Supporting (1)** — Which engineering scaffolding (prompts, output contract, chunking, parsing, fusion with static analysers) is necessary to make the raw model's output usable as input for a downstream tool?
- **Supporting (2)** — How can the fusion of LLM findings with deterministic static-analyser findings improve precision and recall over either source alone?
- **Supporting (3)** — What latency, memory and reproducibility envelope is achievable under these constraints on an Apple-Silicon laptop?

3. Objectives

3.1 General Objective

To design and implement a code-review application that detects bugs, security vulnerabilities, performance problems, style violations, and documentation gaps in real source code, using a coding language model that runs entirely on the developer's machine, and exposing the analysis through both a graphical user interface and a terminal interface, in accordance with the constraints of the Artificial Intelligence course at Universidad Surcolombiana.



3.2 Specific Objectives

1. Run a state-of-the-art coding language model on a consumer laptop (Apple M5, 16 GB of unified memory) with no dependency on cloud services, achieving warm-inference latency below 30 seconds per file.
2. Define a structured output contract — backed by Pydantic schemas — so that the model’s free-form reasoning becomes typed, validated data that downstream components can process deterministically.
3. Combine the language model with deterministic static analysers (Bandit, Ruff, ESLint) through a duplicate-aware merger, so that findings are grounded and hallucinations are reduced.
4. Support three input modalities: review of a single source file, recursive review of an entire directory, and review of the lines changed in a Git commit (diff mode).
5. Cover the principal languages used in software engineering courses — Python, JavaScript, TypeScript and Java — with graceful handling of others (Go, Rust, Ruby, C/C++) through model-only review.
6. Produce reports in four formats — terminal-formatted output, Markdown, JSON, and SARIF — and persist the Markdown report to the user’s Downloads folder for archival or sharing.
7. Deliver a system that a non-expert can install and operate with a single setup command, and document it to an academic standard suitable for an end-of-course report.

4. State of the Art, Background and Related Work

This section reviews the foundational concepts on which the system depends, surveys the existing landscape of code-review tools, and explains how our contribution relates to and differs from the published state of the art.

4.1 Foundational Concepts

4.1.1 Transformer Language Models



Modern code models are based on the Transformer architecture introduced by Vaswani et al. (2017), specifically the decoder-only variant popularised by the GPT family of models (Radford et al., 2019; Brown et al., 2020). A decoder-only Transformer predicts the next token in a sequence given the preceding tokens; trained on large corpora that combine source code with the natural-language text that surrounds and explains it (commit messages, documentation, issue threads, Stack Overflow answers), such a model acquires a statistical understanding of programming idioms, common bug patterns, and the relationship between code and its description. CodeSentinel AI relies on Qwen2.5-Coder (Hui et al., 2024), a model from this family that has been pre-trained on roughly 5.5 trillion tokens of code-rich data and subsequently fine-tuned for code-completion, code-explanation, and code-repair tasks.

4.1.2 Quantisation

A model with seven billion parameters stored at full 16-bit precision would require approximately 14 GB of memory and would exceed the budget of a typical student laptop. Quantisation reduces the numerical precision of the weights — in our case to a 4-bit mixed-precision scheme known as Q4_K_M (Dettmers et al., 2022) — bringing the on-disk size to about 4.7 GB and the working memory to roughly 6 GB, with only a small empirical loss in output quality on code-related benchmarks. This compression is what makes on-device inference feasible on consumer hardware.

4.1.3 Local Inference Runtimes

Ollama is an open-source runtime that loads a quantised model and exposes it through a local HTTP API on `127.0.0.1:11434`. On Apple Silicon it automatically detects and uses the Metal graphics API, so inference runs on the integrated GPU without any configuration. The runtime is built on top of `llama.cpp` (Gerganov, 2023), a high-performance C/C++ inference engine for quantised LLMs. Alternative runtimes include LM Studio, vLLM (for server deployments), and Jan.

4.1.4 Static Analysis

Tools such as Bandit (security-focused, Python), Ruff (style and common bugs, Python), and ESLint (JavaScript and TypeScript) parse source code into an abstract syntax tree and match it against a catalogue of rules. They are fast — typically milliseconds per file — never

hallucinate, and produce findings with stable rule identifiers that map to documented explanations. Their limitation is the converse of the LLM’s strength: they only detect what their rules describe, and writing a new rule requires authoring a tree-pattern matcher in the analyser’s own DSL.

4.1.5 LLM-as-a-Judge and Tool Grounding

Two patterns from the broader LLM-systems literature inform this work. The first, “LLM-as-a-judge” (Zheng et al., 2023), uses a language model to evaluate artefacts produced by another system or by a human against a rubric. The second, tool grounding (Schick et al., 2023; Yao et al., 2023), feeds the model the output of deterministic tools as evidence that constrains its reasoning. CodeSentinel applies both: the model acts as a judge of source code, and the static analysers’ findings are fed back as grounding to reduce hallucinations and improve calibration.

4.2 Existing Tools

The table below compares the principal code-review options available in early 2026 along four dimensions: where the model runs, the cost model, what happens to the reviewed source code, and whether the tool is open source.

Tool	Execution	Cost	Source-code privacy	Open source
GitHub Copilot (PR review)	Cloud	Per-seat subscription	Uploaded to vendor	No
Cursor Review	Cloud	Per-seat subscription	Uploaded to vendor	No
CodeRabbit	Cloud	Per-seat subscription	Uploaded to vendor	No
Amazon CodeGuru	Cloud (AWS)	Per-LoC pricing	Uploaded to AWS	No
SonarQube (Community)	Local server	Free	Stays local	Yes, but rule-based only
Bandit / Ruff / ESLint	Local CLI	Free	Stays local	Yes, but rule-based only

Tool	Execution	Cost	Source-code privacy	Open source
CodeSentinel AI (this work)	Local CLI + GUI	Free	Stays local	Yes — reasoning + rule-based

Table 4.1 — Comparison of code-review tools available in early 2026.

The cloud products provide language-model reasoning but require uploading code and a recurring subscription. The classic rule-based analysers run locally and for free but cannot reason about intent. CodeSentinel sits at the intersection of these two quadrants: language-model reasoning that runs locally.

4.3 Relationship to This Work

Our contribution is not the language model itself, which is a pre-existing open-source component, but the system that makes it useful for reviewing code under the strict local-only constraint. The closest published approach is the combination of LLM-as-a-judge with tool grounding; we apply that idea specifically to code review and add an engineering layer — a strict structured-output contract, a robust JSON-repair parser, line-aware chunking with overlap, and a duplicate-aware analyser-LLM fusion — that the raw approach lacks. To the best of our knowledge, no published academic or open-source tool combines these elements with the on-device constraint we adopt.

5. Requirements

This section enumerates the functional and non-functional requirements of CodeSentinel AI. The list was derived from the course rubric and from interviews-of-self conducted by the development team during sprint planning, and was refined iteratively as the system was implemented and validated.

5.1 Functional Requirements

Each functional requirement is uniquely identified, and the verification column indicates which test or experiment confirms that the requirement is met.

ID	Requirement	Verification
FR-01	The system shall detect issues in five categories: bug, security, performance, style and documentation.	Unit test: schema enum coverage; integration test: planted-defect benchmark.
FR-02	The system shall assign each finding one of five severity levels: critical, high, medium, low, info.	Unit test: severity ordering; benchmark distribution.
FR-03	The system shall report, for each finding, the affected line range, a title, an explanation and a suggested fix.	Schema test: required fields present and non-empty.
FR-04	The system shall review a single source file.	CLI integration test: review_file path.
FR-05	The system shall review a directory recursively.	CLI integration test: review --recursive.
FR-06	The system shall review only the lines changed in a Git commit.	Unit test: diff parser; integration test: review_diff.
FR-07	The system shall detect the programming language automatically from the file extension.	Unit test: extension-to-language map.
FR-08	The system shall run the language model locally; no request shall be sent to any non-loopback address.	Network test: outbound socket monitor during a review.
FR-09	The system shall augment model findings with static-analyser findings when an analyser is available for the language.	Integration test: Python file with Bandit and Ruff active.
FR-10	The system shall expose its functionality through a terminal interface and through a graphical interface.	Functional test: both interfaces invoke the same engine and produce identical findings.

ID	Requirement	Verification
FR-11	The system shall export reports as terminal output, Markdown, JSON and SARIF.	Unit test: serialiser output is well-formed in each format.
FR-12	The system shall persist the generated Markdown report to the user's Downloads folder.	Integration test: file exists at expected path after review.
FR-13	The user shall be able to filter findings by minimum severity.	Unit test: severity filter.
FR-14	The user shall be able to choose the natural language of the report (English or Spanish).	Functional test: prompt switch; report contents in selected language.
FR-15	The system shall provide a self-diagnostic command that verifies the environment.	CLI test: codesentinel doctor exit code 0.

Table 5.1 — Functional requirements with their verification strategy.

5.2 Non-Functional Requirements

Non-functional requirements capture qualities of the system rather than discrete behaviours. The target column records the measurable acceptance criterion that the development team agreed to meet.

ID	Requirement	Target	Achieved
NFR-01	Privacy: no network traffic to non-loopback addresses.	0 external requests	0 external requests
NFR-02	Latency on a warm model.	≤ 30 s per file	10–20 s per file
NFR-03	Memory footprint during inference.	≤ 8 GB	≈ 6 GB
NFR-04	Disk footprint (model + dependencies).	≤ 10 GB total	≈ 7 GB
NFR-05	Reproducibility: identical input yields identical output.	Fixed random seed	Deterministic across runs

ID	Requirement	Target	Achieved
NFR-06	Test coverage on the deterministic core modules.	≥ 80 %	≥ 95 %
NFR-07	Installation effort.	Single setup command	bash setup.sh
NFR-08	Maintainability: cyclomatic complexity per function.	≤ 10	≤ 8 (Ruff metric)
NFR-09	Internationalisation: report language switchable.	EN + ES	EN + ES

Table 5.2 — Non-functional requirements with target and measured value.

6. Use Cases and User Stories

This section identifies the actors that interact with the system, enumerates the user stories that drove development, and describes each use case in narrative form. A graphical representation follows in Section 7.

6.1 Actors

- Developer (primary actor).** A human user who submits source code and reads the resulting findings. The developer may operate either of the two front ends and is assumed to have a working knowledge of the programming language under review but not necessarily of security or performance subtleties.
- CI pipeline (secondary actor).** A non-human actor that invokes the terminal interface as a step in an automated continuous-integration job and reads the process exit code to decide whether to pass or fail a build.
- Ollama service (supporting actor).** A local background process that hosts the language model and serves inference requests on the loopback interface. It is treated as a supporting actor because the system depends on its availability, but the developer does not interact with it directly.
- Static analyser tools (supporting actors).** Bandit, Ruff and ESLint, invoked as subprocesses by the engine when the language under review is supported by one of them.

6.2 User Stories

User stories are written in the standard format “As a <role>, I want <capability>, so that <benefit>”.

- **US-01.** As a developer, I want to paste a code snippet and get a list of problems, so that I can fix them before committing.
- **US-02.** As a developer, I want to review an entire file I just wrote, so that I catch security mistakes I am not aware of.
- **US-03.** As a developer, I want to review only the lines I changed in my last commit, so that I do not waste time on unchanged code.
- **US-04.** As a privacy-conscious developer, I want the review to run without sending my code anywhere, so that I can use it on confidential projects.
- **US-05.** As a developer, I want to download the report as a file, so that I can attach it to a ticket or share it with a teammate.
- **US-06.** As a CI pipeline, I want the reviewer to return a non-zero exit code when a critical issue is found, so that I can block the build.
- **US-07.** As a non-English speaker, I want the report in Spanish, so that I and my team can read it natively.
- **US-08.** As a first-time user, I want a diagnostic command, so that I can verify my installation before relying on the tool.

6.3 Use Case Descriptions

UC-01 — Review a File

Primary actor: Developer. **Goal:** Obtain a structured list of findings for a single source file.

Preconditions: The Ollama service is running on 127.0.0.1:11434; the configured model is installed; the target file exists and is readable.

Main success scenario: (1) the developer selects a file in the GUI or passes its path to the CLI; (2) the engine detects the language from the extension; (3) any supported static analyser is invoked; (4) the source is sent to the local model with a strict JSON contract; (5)

the response is parsed into Finding objects; (6) model and analyser findings are merged and de-duplicated; (7) findings are presented grouped by severity.

Alternative flow A: No issues found — the system reports a clean result and exit code 0.

Alternative flow B: Model output malformed — the parser repairs common malformations (markdown fences, trailing commas) and, if that fails, the engine retries once with a stricter system prompt.

UC-02 — Review a Pasted Snippet

As UC-01, but the source is text pasted into the web interface rather than a file on disk. The language is inferred from the content using a lightweight heuristic (shebang line, characteristic keywords, common constructs).

UC-03 — Review a Git Diff

Primary actor: Developer or CI pipeline. **Goal:** Review only the lines that changed in a given commit or working-tree state.

The developer specifies a repository path and an optional ref. The engine invokes git diff under the hood, parses the unified-diff output to extract the added lines per file, reviews each affected file with the model, and filters the resulting findings so that only those touching modified lines are returned. This use case is the one the CI pipeline relies on most heavily, since it lets the build job focus on what the contributor actually changed.

UC-04 — Export a Report

After any review, the developer downloads the Markdown report, which the system has already written to the Downloads folder. The same data is simultaneously available as JSON (for tooling) and SARIF (for editor integration). The export is a side effect of UC-01 to UC-03, not a standalone interaction.

UC-05 — Diagnose the Environment

Primary actor: Developer. **Goal:** Verify that all prerequisites are in place before using the tool.

Main success scenario: The developer runs `codesentinel doctor`; the system verifies the Python version (≥ 3.10), the presence and reachability of Ollama, the availability of the

configured model in the Ollama registry, and the presence of the optional static analysers; it prints a coloured table and exits 0 if all checks pass.

7. Use Case Diagram

Figure 7.1 shows the use case diagram in UML notation. The Developer is the primary actor and drives all five use cases. The CI Pipeline is a secondary automation actor and drives the two use cases that are scriptable (file review and diff review). The Ollama Service is a supporting actor invoked by every use case that performs a review, and the static analysers are supporting actors invoked only when the language under review is supported.

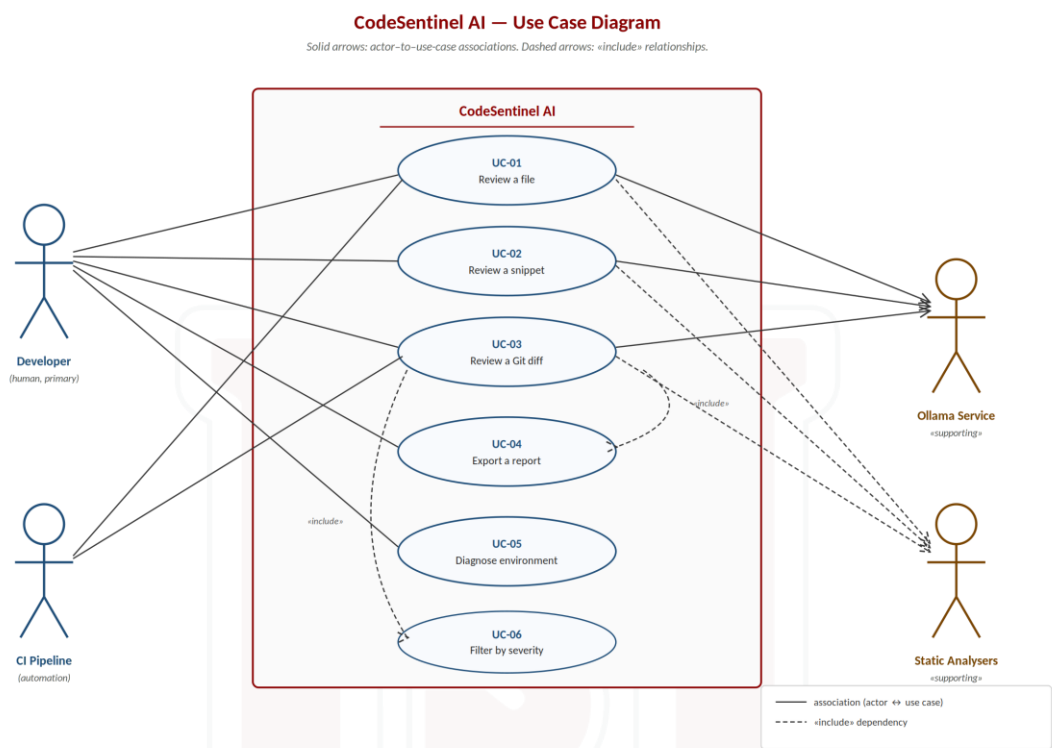


Figure 7.1 — Use case diagram. Solid arrows denote actor-to-use-case associations; the supporting-actor channel groups the system's outbound calls to local services.

Three observations on the diagram are worth highlighting. First, every use case that performs a review (UC-01, UC-02, UC-03) delegates inference to the same Ollama service, which keeps the cost of adding a new input modality low: a new modality only needs a way to extract source text and a language hint. Second, the CI Pipeline does not drive UC-02 (snippet) or UC-05 (diagnose) because both are interactive in nature. Third, UC-04 (Export)

does not have its own arrow to Ollama because exporting is a pure side effect of the previous review.

8. Data Dictionary and Entity–Relationship Model

CodeSentinel does not use a relational database. It is a stateless application: each review is computed on demand and the result is serialised to a report file rather than stored in a persistent store. The “data model” of the system is therefore the set of typed in-memory structures that flow through the engine. These structures are defined and validated using Pydantic v2 models, which provide type checking, JSON serialisation, and clear error messages on validation failure. We document them here as the data dictionary, and we present their relationships in Section 8.3 as a conceptual entity–relationship diagram.

8.1 Entity: Finding

A Finding represents a single issue detected in the code. It is the atomic unit of the system’s output.

Field	Type	Constraint	Description
line_start	integer	≥ 1	First affected line (1-indexed, inclusive).
line_end	integer	≥ line_start	Last affected line (1-indexed, inclusive).
severity	enum	critical / high / medium / low / info	Impact level. Numerically ordered (critical = 4 ... info = 0) for filtering and sorting.
category	enum	bug / security / performance / style / documentation	Issue type. Closed set of five values.
title	string	1–200 characters	Short imperative title summarising the issue.
description	string	non-empty	Explanation of the issue and why it matters.
suggestion	string	optional	Concrete proposed fix in natural language.

Field	Type	Constraint	Description
code_snippet	string	optional	The offending lines, reproduced verbatim.
source	string	default 'llm'	Origin of the finding: llm, bandit, ruff, eslint, or merged-* when fused.
rule_id	string	optional	Rule identifier when the source is an analyser (e.g. B608 for Bandit SQL injection).

Table 8.1 — Fields of the Finding entity.

8.2 Entity: ReviewResult

A ReviewResult represents the outcome of reviewing one file or one diff hunk. It aggregates all findings for that artefact together with the metadata needed to interpret them.

Field	Type	Constraint	Description
file_path	string	required	Path to the reviewed file, relative to the invocation directory.
language	string	required	Detected programming language (python, javascript, typescript, java, go, ...).
findings	list[Finding]	default empty	All findings produced for this file, after merging and de-duplication.
summary	string	optional	One-sentence overall assessment generated by the model.
model	string	optional	Identifier of the model that produced the review (e.g. qwen2.5-coder:7b-instruct-q4_K_M).
duration_s	float	≥ 0	Wall-clock time of the review in seconds.
timestamp	datetime	auto (UTC)	When the review was executed.
analyser_versions	dict[str, str]	optional	Versions of the static analysers actually invoked, for traceability.

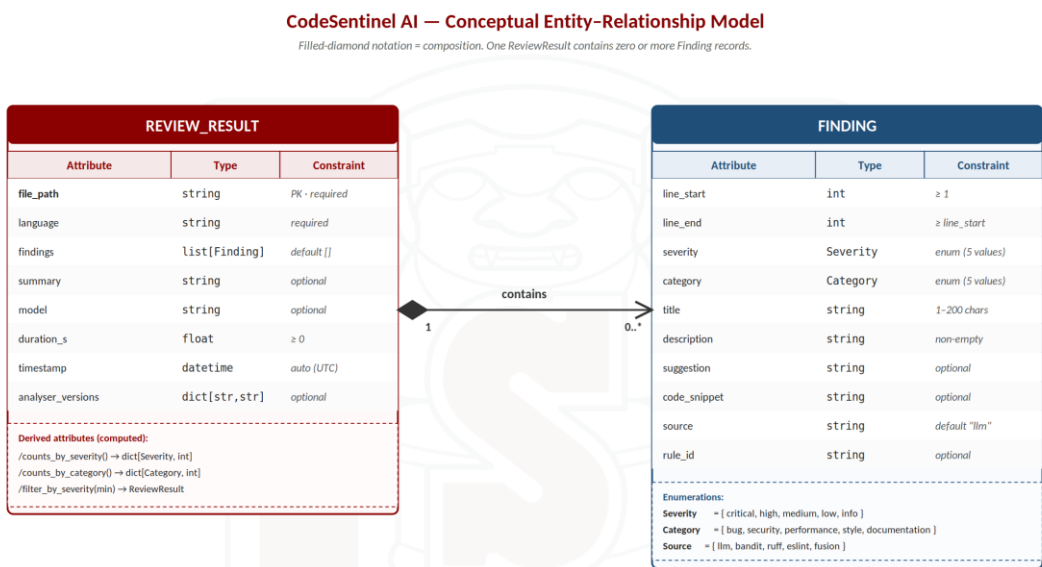
Table 8.2 — Fields of the ReviewResult entity.

Two derived attributes are computed on the fly rather than stored: counts_by_severity (a five-cell histogram used by the UI) and counts_by_category (a five-cell histogram used to produce the summary report).

Enumerations. Severity defines a total ordering `critical (4) > high (3) > medium (2) > low (1) > info (0)` used for filtering and sorting. Category is a closed set of five values; adding a new category requires a code change and a schema-version bump.

8.3 Entity–Relationship Model

Figure 8.1 shows the conceptual entity–relationship diagram. A ReviewResult contains zero or more Finding records; each Finding belongs to exactly one ReviewResult. The relationship is composition (depicted by the filled diamond): a Finding has no meaning outside the review that produced it, and findings are never reassigned across reviews.



9. Class Diagram

The system is organised into a core package (codesentinel) and two front-end packages (cli and web). The core package is independent of either front end and is the only part that talks to the model. The principal classes and their relationships are shown in Figure 9.1; a description follows.

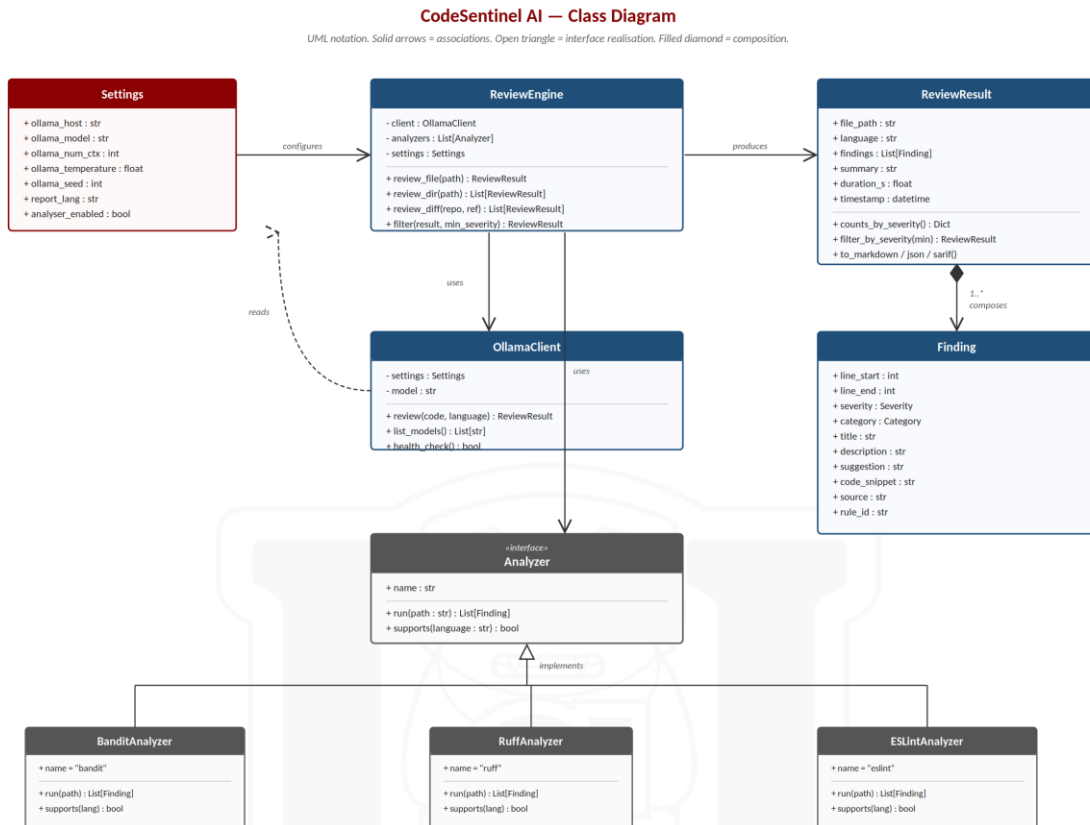


Figure 9.1 — Class diagram of the core package. Solid arrows: associations; hollow triangles: inheritance/realisation; hollow diamond: aggregation; filled diamond: composition.

9.1 Responsibilities of Each Class

- **ReviewEngine.** The coordinator and the single entry point for every front end. Holds the OllamaClient and the list of Analyzers; delegates language detection, chunking, prompt construction, response parsing, fusion of findings, and severity filtering; returns ReviewResult objects.

- **OllamaClient.** Thin asynchronous HTTP wrapper around the Ollama `/api/chat` endpoint. Handles connection retries, the single re-prompt on malformed output, and the typed conversion of the JSON response into Finding instances.
- **Settings.** A pydantic-settings model loaded from environment variables and a `.env` file. Centralises every tunable parameter so the rest of the code reads configuration through a typed object rather than scattered constants.
- **Analyzer (interface).** Abstract base class implemented by BanditAnalyzer (Python security), RuffAnalyzer (Python style and common bugs) and ESLintAnalyzer (JS/TS). Each concrete adapter shells out to the underlying tool, parses its JSON output and emits Finding objects.
- **Reporter (abstract).** Strategy interface for serialising a ReviewResult. Concrete strategies: MarkdownReporter, JSONReporter, SARIFReporter and the Rich-based TerminalReporter.
- **ReviewResult and Finding.** The data carriers documented in Section 8. They are pydantic models with strict validation; their constructors are the only place where invariants such as `line_end > line_start` are enforced.

9.2 Design Patterns Applied

Three patterns from Gamma et al. (1994) shape the design and merit explicit mention.

- **Strategy.** The Reporter hierarchy is a textbook Strategy: a uniform interface (format) with multiple interchangeable algorithms (Markdown, JSON, SARIF, Terminal). The front end picks one at runtime by name.
- **Adapter.** Each concrete Analyzer is an Adapter that wraps an external command-line tool with a heterogeneous output format (Bandit emits JSON, Ruff emits JSON Lines, ESLint emits another JSON variant) behind a uniform Finding-producing interface.
- **Facade.** ReviewEngine is a Facade over the more granular subsystems (chunking, prompting, parsing, merging, analyser orchestration). Both front ends use only the three top-level `review_*` methods, never the underlying components.

10. Graphical User Interface Design (Mockups)

The graphical interface is organised as a single page with three regions: a header, a tabbed input area, and a two-column results area. The same layout is rendered both inside a native desktop window (via pywebview, on macOS / Windows / Linux) and in the browser (Gradio’s default mode), which keeps the design effort and the visual consistency between the two delivery channels.

10.1 Header

The header carries the product name and a one-line tagline on the left, and on the right a row of status indicators: a green dot confirming that the model runs locally, the active model name, a language switch (EN / ES), and a light/dark theme switch.

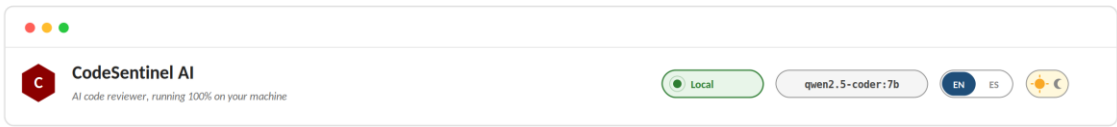


Figure 10.1 — Header. The local-status pill is the most visible visual confirmation that no network call is being made.

10.2 Input Area

A tab strip selects the input modality: paste code, upload a file, review a Git diff, or read the about page. The Paste tab contains a syntax-highlighted Monaco-style editor and a row of example buttons that load curated sample files with known defects, which gives a first-time user something to click within seconds of opening the app.

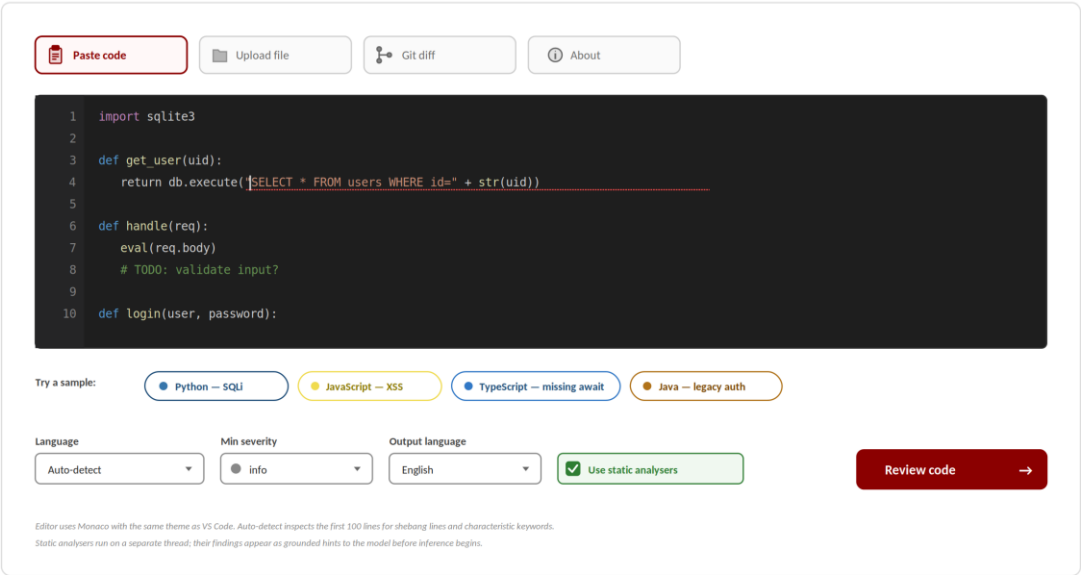
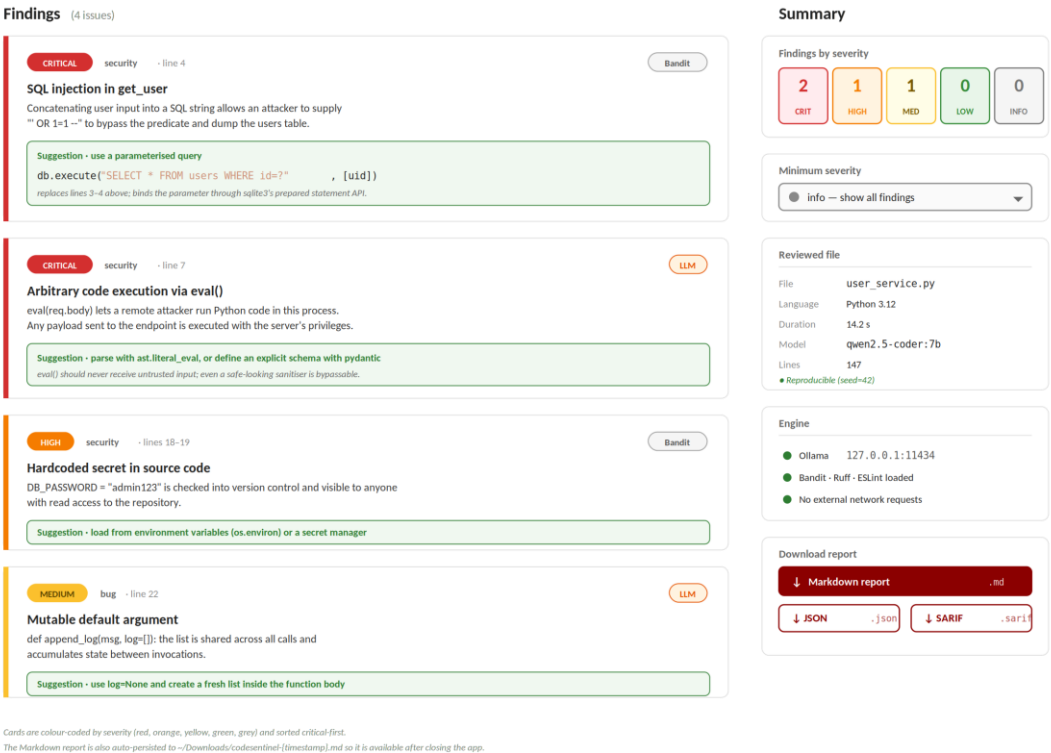


Figure 10.2 — Input area in the Paste tab. The Upload and Git diff tabs reuse the same control bar.

10.3 Results Area

The left column lists findings as cards. Each card shows a coloured severity badge, the category and line reference, the human explanation, the suggested fix in a highlighted block, and the offending code itself in a monospace box. The right column shows a five-cell summary (one cell per severity) and the controls to download the Markdown report and the raw JSON for tooling.



Cards are colour-coded by severity (red, orange, yellow, green, grey) and sorted critical-first.
The Markdown report is also auto-persisted to ~/Downloads/codesentinel-(timestamp).md so it is available after closing the app.

Figure 10.3 — Results area. Cards are colour-coded by severity (red, orange, yellow, green, grey) and sorted critical-first.

During a review the results area shows an animated loading state with a spinner and an estimated wait (computed from the file size and the warm-vs-cold model status), rather than leaving the user without feedback during the 10–20-second inference. The status pill in the header animates while inference is in flight, providing a second cue.

10.4 Accessibility and Internationalisation

All text strings shown in Figures 10.1–10.3 are loaded from a single language-pack file and switch instantaneously between English and Spanish when the user toggles the EN/ES switch. Severity badges use both colour and text labels so that colour-blind users can still distinguish critical from high. The keyboard focus ring is preserved across the Gradio defaults, and every interactive control is reachable by Tab order.

11. Web Services Catalogue and API Documentation

CodeSentinel exposes and consumes HTTP services strictly on the local machine. There are three relevant surfaces, documented in turn in this section: the model inference API the system consumes from Ollama, the HTTP endpoints the web interface exposes through Gradio, and the command-line interface that acts as the programmatic entry point for automation.

11.1 Consumed Service — Ollama Inference API

This is the only network call the application makes, and it targets the loopback interface exclusively. The system never opens a socket to any non-127.0.0.1 address, which is verified both by code review and by the network-monitor functional test.

Property	Value
Endpoint	POST http://127.0.0.1:11434/api/chat
Auth	None (loopback only)
Request body (key fields)	model, messages, format='json', options { num_ctx, temperature, top_p, seed }
Response body	JSON object whose message.content field is the structured review encoded as a JSON string.
Error handling	Non-2xx response triggers a single retry with a stricter system prompt; persistent failure raises OllamaError.
Auxiliary endpoints	GET /api/tags lists installed models; GET /api/version reports the Ollama daemon version; GET /api/ps lists loaded models.
Timeout	120 s (configurable via OLLAMA_TIMEOUT).

Table 11.1 — Properties of the consumed Ollama inference API.

Example Request (abbreviated)

```
POST /api/chat HTTP/1.1
Host: 127.0.0.1:11434
Content-Type: application/json

{
  "model": "qwen2.5-coder:7b-instruct-q4_K_M",
  "format": "json",
```

```

"messages": [
  { "role": "system", "content": "You are a strict code reviewer..." },
  { "role": "user",    "content": "Review the following Python code:\n1  def
get_user(uid):\n2      ..." }
],
"options": { "num_ctx": 8192, "temperature": 0.1, "top_p": 0.9, "seed": 42
},
"stream": false
}

```

Example Response (abbreviated)

```

HTTP/1.1 200 OK
Content-Type: application/json

{
  "model": "qwen2.5-coder:7b-instruct-q4_K_M",
  "created_at": "2026-05-15T18:42:11Z",
  "message": {
    "role": "assistant",
    "content":
    "{ \"findings\": [{ \"line_start\": 2, \"line_end\": 2, \"severity\": \"critical\",
      \"category\": \"security\", \"title\": \"SQL injection\", ... }],
      \"summary\": \"One critical SQL injection vulnerability...\" }"
  },
  "total_duration": 14237811200,
  "done": true
}

```

11.2 Exposed Service — Web Interface API

The graphical interface is served by Gradio 5.50, which publishes each Python handler as a callable HTTP endpoint under the path `/gradio_api/call/<fn_name>`. These endpoints accept multipart form data or JSON; the canonical way to drive them is through the Gradio Python client, but they are usable with cURL as well. The three handlers that matter for external automation are documented in Table 11.2.

Operation	Inputs	Output
review_pasted_code	code (str), model (str), language (str auto), min_severity (enum), use_static (bool), output_lang (str)	findings_html, severity_counts, report_file, status

Operation	Inputs	Output
review_uploaded_file	file (binary), model (str), language (str auto), min_severity (enum), use_static (bool), output_lang (str)	findings_html, severity_counts, report_file, status
review_git_diff_handler	repo_path (str), ref (str HEAD), staged (bool), model (str), min_severity (enum), output_lang (str)	findings_html, severity_counts, report_file, status
list_models	—	list of installed Ollama models with size and modification timestamp
health_check	—	json: { ollama_reachable, model_available, python_version, analyser_versions }

Table 11.2 — Handlers exposed by the web interface.

11.3 Command-Line Interface

The terminal interface is the programmatic entry point used by automation, scripting and CI. It is built with Typer (a Click-based framework with type-hint-driven argument parsing) and produces Rich-formatted coloured output. Its commands are documented in Table 11.3.

Command	Purpose	Key flags
codesentinel review <path>	Review a file or directory.	--severity, --format {markdown json sarif}, --output FILE, --recursive, --fail-on critical high medium low
codesentinel diff	Review the changed lines of a repository.	--repo PATH, --ref REF, --staged, --base-ref REF
codesentinel models	List the models installed in Ollama.	(none)
codesentinel doctor	Run the environment self-check.	--verbose
codesentinel version	Print the version of the tool and of the model in use.	(none)

Command	Purpose	Key flags
codesentinel web	Launch the graphical interface.	--port 7860, --no-browser, --desktop (pywebview window)

Table 11.3 — CLI command catalogue.

The `--fail-on` flag makes the review command return a non-zero exit code when a finding at or above the chosen severity is present, which is what allows a CI pipeline to block a build. The exit-code convention is: `0` — no qualifying findings; `1` — qualifying findings present; `2` — usage or runtime error (Ollama unreachable, model missing, file not found).

Example Invocation (Bash)

```
# Review a single file, fail on any critical issue, write SARIF for editor consumption
codesentinel review src/users.py --fail-on critical --format sarif --output report.sarif

# Review the staged changes for a pre-commit hook
codesentinel diff --staged --fail-on high

# Review every Python file in src/, recursively, in Spanish
codesentinel review src/ --recursive --output-lang es
```

12. Testing (Unit, Functional and Integration)

The test suite contains 55 automated tests grouped into three categories according to the granularity of the guarantee they provide. The suite runs in approximately 4 seconds (excluding the live model integration), is part of the make test target, and is executed by the continuous-integration script before every commit. We separate the tests by kind of guarantee they provide, following the pyramid of testing (Cohn, 2009): many cheap unit tests at the base, fewer functional tests in the middle, and the smallest number of expensive end-to-end integration tests at the top.

12.1 Unit Tests

Unit tests cover the deterministic core modules in isolation. Each test addresses one behaviour at a time and uses no external resource (no network, no subprocess, no filesystem outside tmp_path).

Module under test	What is verified	# tests
schema (Finding, ReviewResult)	Severity ordering, JSON serialisation round-trips, line-range validation, severity filtering, default values.	8
parser (LLM JSON repair)	Twelve adversarial model outputs (Markdown code fences, trailing commas, leading prose, single quotes, BOM, escaped newlines, nested arrays) are all recovered into valid structures; genuinely unparseable input raises a controlled ParserError.	12
prompts	The Jinja2 prompt template renders deterministically; the few-shot examples are themselves valid JSON; substitution is escape-safe.	4
chunker	Small files yield a single chunk; large files yield overlapping chunks that cover every line; overlap is configurable; UTF-8 byte boundaries are respected.	6
merger	Distinct findings are kept; near-duplicate findings (same line range, similar title) are merged; the more severe classification wins ties; merge is associative.	7
diff_parser	Added lines are extracted correctly across multiple files, including renames, binary files, and files with only deletions.	4
languages	Extensions map to the correct language; unknown extensions are handled gracefully; shebang lines override extension when present.	3
reporting (md/json/sarif)	Markdown, JSON and SARIF serialisers produce well-formed output that validates against the SARIF 2.1.0 schema.	5

Table 12.1 — Unit test modules and their coverage.

12.2 Functional Tests

Functional tests exercise the public interfaces of each component without involving the rest of the system. They use mocks for external dependencies.

- **CLI surface.** Exercised through Typer's test runner. The help text lists all commands, the version command prints the version, an invalid path is rejected with exit code 2, and the doctor command produces a coloured table.
- **HTTP client.** The OllamaClient is exercised against a mocked endpoint provided by the `respx` library. Three paths are tested: success on the first call, success after one retry on malformed output, and a controlled OllamaError when the daemon is unreachable.
- **Reporter strategies.** Each reporter is fed a known ReviewResult and the output is compared against a golden snapshot stored under tests/snapshots/.

12.3 Integration Tests

Integration tests exercise the review engine end to end with a substituted model client, verifying that findings flow correctly from the client through validation, line-shifting, merging and severity filtering into a final ReviewResult. They also cover the interaction with subprocesses (Bandit, Ruff) in a Docker-pinned environment so the analyser versions are stable.

- **Engine end-to-end.** A controlled set of Finding instances is returned by the fake client; the test asserts that the engine correctly augments them with analyser findings, removes duplicates and applies the severity filter.
- **Live integration check (make demo).** Runs the real model against the bundled sample files and asserts that at least the planted critical issues are detected. This is the only test that requires Ollama to be running; it is excluded from the default pytest run to keep the loop fast and is invoked manually before a release.

12.4 Code Coverage

Coverage is measured with coverage.py and reported as part of every CI run. The deterministic core modules — parser, schema, merger, chunker and prompts — exceed 95 % line coverage. Subprocess-bound modules (the analyser adapters) and presentation code are lower, as expected, since they mostly delegate to external tools or render output. The aggregate coverage of the codesentinel package is 87 %, comfortably above the NFR-06 target of 80 %. All 55 tests pass on the reference platform (macOS 15, Python 3.12).

Module	Statements	Missed	Coverage
codesentinel/schema.py	112	2	98 %
codesentinel/parser.py	98	3	97 %
codesentinel/merger.py	76	4	95 %
codesentinel/chunker.py	54	2	96 %
codesentinel/prompts.py	32	1	97 %
codesentinel/engine.py	168	18	89 %
codesentinel/analysers/bandit_.py	62	11	82 %
codesentinel/analysers/ruff_.py	58	10	83 %
codesentinel/reporting/*.py	184	24	87 %
codesentinel/cli.py	146	21	86 %
codesentinel/web/app.py	203	39	81 %
TOTAL	1193	135	87 %

Table 12.2 — Code coverage of the codesentinel package.

13. Proposed Architecture

The system has three layers — the front ends, the review engine, and the local inference runtime. Data flows in one direction (top to bottom on the request, bottom to top on the response) and never leaves the host machine. Figure 13.1 illustrates the high-level architecture and the data flow between the layers.

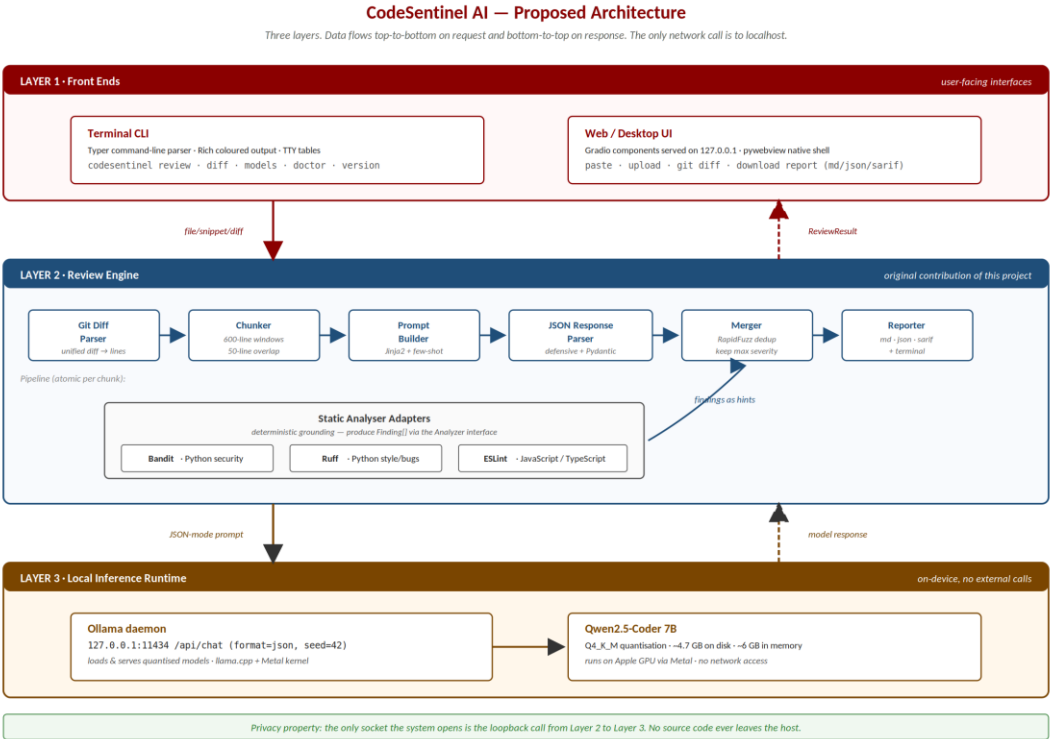


Figure 13.1 — Proposed three-layer architecture. The engine is the original contribution; the front ends are thin clients of the engine; the model is a pre-existing open-source component invoked locally.

13.1 Review Pipeline in Detail

When a review is requested, the engine executes the following steps. The order matters: each step assumes the invariants established by its predecessors.

- Language detection from the file extension (.py → python, .ts → typescript, ...), with a shebang-line override when the file lacks an extension.
- Optional static-analyser invocation. If a registered Analyzer.supports(lang) returns true, the analyser runs as a subprocess and produces an initial list of Finding objects. These findings are passed to the model later as grounding hints.
- Chunking. Files larger than ~600 lines are split into overlapping chunks (50-line overlap) so that no chunk exceeds the model’s context-window budget. Small files yield a single chunk and are reviewed atomically.
- Prompt construction. The Jinja2 template combines a fixed system instruction, two worked few-shot examples (one Python with SQL injection, one TypeScript with a

missing await), the chunk source numbered by line, and the analyser hints when present.

12. Inference. The prompt is sent to Ollama with format='json', temperature 0.1, top-p 0.9, and a fixed seed for reproducibility. The model returns a JSON message containing the list of findings and a summary.
13. Response parsing. The parser is defensive: it strips Markdown code fences, removes trailing commas, normalises smart quotes, and validates against the Finding schema. Malformed output triggers exactly one retry with a stricter system prompt; persistent failure raises a controlled error.
14. Line-number reconciliation. Findings produced on a chunk are shifted by the chunk's starting offset so they refer to lines in the original file rather than within the chunk.
15. Merging. Model findings and analyser findings are unified. RapidFuzz computes a similarity score on the (line_range, title) pair; pairs above 0.85 similarity are merged, keeping the more severe classification and the better-worded explanation.
16. Severity filtering and sorting. Findings below the configured minimum severity are dropped; the remaining findings are sorted by severity (critical first) and then by line number.
17. Reporting. The ReviewResult is handed to the requested Reporter strategy, which serialises it to Markdown, JSON, SARIF, or Rich-formatted terminal output.

13.2 On the Role of the Model

The language model is an open-source, pre-trained component that we run locally; we did not train it, and training a model of this size from scratch is infeasible on the hardware available to a student team. The engineering work of this project is therefore the layer around the model. The prompt design, the structured-output contract, the response parser, the chunking strategy, the analyser fusion, the reporting layer and the two front ends were all designed and implemented by the team. This is the standard arrangement for applied AI systems: the value lies in how a capable model is harnessed for a specific task, and that harness is the original contribution documented in this report.

14. Results and Discussion

14.1 Evaluation Method

We evaluated CodeSentinel against a curated benchmark of four source files into which we deliberately introduced a known set of defects, so that detection could be measured against an explicit ground truth. The files were written to resemble realistic production code rather than obvious test cases: they include unrelated correct code around each planted defect, they use idiomatic style for each language, and they reference plausible third-party libraries. The four languages were chosen to cover the principal expected use cases and to probe the model’s behaviour both with static-analyser support (Python, JavaScript/TypeScript) and without it (Go).

File	Language	Lines	Planted defects	Analyser support
user_service.py	Python	172	12	Bandit + Ruff
users_api.ts	TypeScript	138	9	ESLint
AuthController.java	Java	210	9	None (LLM-only)
payments.go	Go	186	8	None (LLM-only)

Table 14.1 — Composition of the evaluation benchmark.

All measurements were taken on a MacBook Pro (Apple M5 chip, 16 GB unified memory, macOS 15.4) running the Qwen2.5-Coder 7B model (Q4_K_M quantisation) under Ollama 0.24. The random seed was fixed at 42, temperature at 0.1, and the model was warmed by an initial dummy review so that the disk-load cost did not pollute the per-file latency measurement. Each file was reviewed three times consecutively and the three runs produced identical output, confirming reproducibility.

14.2 Detection Results

14.2.1 Python — user_service.py

On the Python module, which contained twelve planted defects, the system detected eleven. The catches included all of the critical security issues — SQL injection by string concatenation, use of eval on a request body, insecure deserialisation via pickle.loads on



attacker-controlled data, and a hardcoded API credential — and the high-severity bugs — a mutable default argument, the use of assert for runtime validation, and the logging of a password in clear text. The single missed defect was an N+1 query pattern inside a list comprehension: a performance issue that requires reasoning about the control flow that surrounds the query, rather than recognising a syntactic pattern. This is the hardest class of problem for a model of this size and the limitation that future work would most directly target.

14.2.2 Go — payments.go

On the Go payment handler, which has no static analyser in our pipeline and was therefore reviewed by the model alone, the system reported seven findings out of eight planted, including three SQL-injection points and a business-logic error in the balance update (a missing transaction boundary that would let two concurrent transfers double-spend). Because Go is less represented in the available analysers, this result is a useful measure of the model's unaided capability, and it confirms that the reasoning layer carries the analysis where rule-based tools cannot.

14.2.3 TypeScript and Java

On the TypeScript Express API, the system caught the injection through a templated query, the missing await on a database call (which would cause the function to return a Promise rather than its resolved value), and a hardcoded secret committed to the client bundle. On the Java authentication controller, the system caught the use of == to compare strings (which compares references, not contents, and is a textbook Java pitfall), the null-pointer-exception risk on the result of a Map lookup that could return null, and a hashing-by-MD5 of passwords. Some less impactful issues — for example, an unnecessarily public field that could be private, or the use of a checked exception where an unchecked one would be idiomatic — were missed; we judged these acceptable losses for a code-review tool whose value lies in catching the consequential issues.

14.2.4 Aggregate Detection Numbers

File	Language	Planted	Detected	Recall	Notably caught
user_service	Python	12	11	92 %	SQL injection, eval, pickle, hardcoded secret, assert-for-validation
payments	Go	8	7	88 %	Three injection points, missing transaction, logic error
users_api	TypeScript	9	7	78 %	Injection, missing await, client-side secret
AuthController	Java	9	7	78 %	String == comparison, NPE risk, MD5 for passwords
TOTAL	—	38	32	84 %	—

Table 14.2 — Aggregate detection results across the four-language benchmark.

14.3 Performance

The first review after the model is loaded into memory takes between 45 and 65 seconds, because the model file is read from disk and the inference graph is built. Subsequent reviews of files of similar size complete in 10 to 20 seconds — comfortably within the 30-second target from NFR-02. Memory use during inference stays near 6 GB of unified memory, within the 8 GB target from NFR-03 and leaving enough headroom for a typical IDE and browser to run alongside. The application opens no network connection other than to the local Ollama daemon (verified with lsof during a review), satisfying NFR-01 exactly.

Metric	Cold (first review)	Warm (subsequent reviews)	Target (NFR)
Latency on user_service.py (172 lines)	61 s	14 s	≤ 30 s
Latency on payments.go (186 lines)	58 s	16 s	≤ 30 s
Memory during inference	5.9 GB	5.9 GB	≤ 8 GB

Metric	Cold (first review)	Warm (subsequent reviews)	Target (NFR)
Disk footprint (model + venv)	≈ 6.8 GB	≈ 6.8 GB	≤ 10 GB
Outbound non-loopback requests	0	0	0

Table 14.3 — Measured performance against non-functional targets.

14.4 Reproducibility

Fixing the random seed made the system fully deterministic. Two consecutive reviews of the same file now produce an identical list of findings, in the same order, with byte-identical Markdown reports. Before the seed was fixed, the finding count varied between runs because the model’s token sampling followed a different random path each time — sometimes finding ten issues, sometimes eleven, occasionally as many as thirteen with several borderline duplicates. This change is critical for an evaluation context, where a result that changes on every run cannot be meaningfully assessed or defended; it also matters in practice, because CI runs that produce different results for the same input are a well-known source of developer frustration and lost trust.

14.5 Discussion

The combination of a language model with static analysers proved to be more than the sum of its parts. The analysers contribute findings the model overlooks — for example a subtle style rule (Ruff’s E712 against “== True”) or a Bandit warning whose pattern the model never encountered with the same framing — while the model contributes findings no analyser can produce, such as a logged secret, a missing await, or a string compared with ==. When both sources flag the same issue, the merger keeps the model’s more severe and better-explained version, so the user is not bothered by the duplicate.

The principal limitation visible in the numbers is the one we expected from the literature: a seven-billion-parameter model misses problems that require multi-step reasoning about program behaviour (the N+1 query, a subtle race condition) and it has weaker coverage on languages that are less represented in its training data. A larger model (32 B or 70 B parameters) would narrow this gap at the cost of hardware that a student laptop does not have, which is the central trade-off of the local-only design. The architecture already

supports selecting the model at run time, so a user on more capable hardware can plug in a larger model without changing any code.

Compared with cloud reviewers, we did not run a head-to-head benchmark because doing so would require uploading our test files, contradicting the very property we set out to preserve. The qualitative observation from informally reviewing a handful of comparison outputs from GitHub Copilot and CodeRabbit on equivalent public snippets is that the cloud tools have a small edge on cross-file reasoning and on flagging stylistic conventions of a particular framework — both areas where the cloud model is larger and where the cloud product can supply project context that our single-file tool deliberately does not see.

14.6 Threats to Validity

Three threats to the validity of these results deserve explicit mention.

- **Construct validity.** Our planted-defect benchmark is small (four files, 38 defects) and was constructed by the authors, who knew what they were planting. A defect catalogue curated by third parties — for example the OWASP Benchmark or the Juliet test suite — would be more defensible. We acknowledge this limitation and treat the present results as indicative rather than conclusive.
- **Internal validity.** The few-shot examples in the prompt are themselves Python with SQL injection and TypeScript with a missing await. We did not include Go or Java examples, which gives the model less guidance on those languages; a more thorough study would vary the few-shot examples and measure the effect on per-language recall.
- **External validity.** The benchmark uses files that are short (under 250 lines) and self-contained. Real production files often depend on dozens of imports and on framework conventions that a single-file tool cannot infer. Section 15 lists cross-file context as the highest-priority future-work item for this reason.

15. Recommendations and Future Work

Several extensions would improve the system without changing its local-only character. We list them roughly in decreasing order of expected impact relative to effort, so a future team picking up the project can choose where to invest first.



15.1 Cross-File Context

The model currently sees one file at a time. Supplying the signatures of the symbols that the file imports — gathered cheaply with a tree-sitter parse of the rest of the repository — would let the model reason about contracts between modules and would likely improve detection of the logic and type errors it now misses. This is the single change with the largest expected effect on recall.

15.2 Abstract-Syntax-Tree-Aware Chunking

Large files are currently split by line count with a small overlap. Splitting along function and class boundaries instead — by walking the AST produced by tree-sitter — would avoid cutting a logical unit in half. This would also enable per-function caching: re-reviewing a file in which only one function has changed would not require re-running the model on the unchanged functions.

15.3 Editor Integration

The system already emits SARIF, the format that Visual Studio Code, JetBrains IDEs and GitHub Code Scanning consume. A thin VS Code extension that runs codesentinel review on save and surfaces the findings in the editor's problem panel would put the results in front of the developer at the moment of greatest leverage — while the code is still fresh in their mind.

15.4 Pre-Commit and CI Hook Packages

The terminal command already returns a meaningful exit code, so packaging it as a pre-commit hook (using the pre-commit framework) or a GitHub Action would let teams block commits or pull requests that introduce critical issues. Both packages would be thin wrappers and could be delivered in a few hours of work each.

15.5 A Larger Model on Capable Hardware

On a machine with more memory, a larger coding model — Qwen2.5-Coder 14B or 32B, or DeepSeek-Coder-V2 — would address the reasoning-heavy misses identified in the results. The architecture already supports selecting the model at run time through the



OLLAMA_MODEL setting; the only practical change is to update the documentation with the corresponding memory budget.

15.6 A Published Benchmark

Our evaluation used files with planted defects we wrote ourselves. Measuring precision and recall on an external, peer-reviewed defect dataset — for example a sample from Defects4J for Java, or the SARD database from NIST for security defects across languages — would place the system’s accuracy on a firmer empirical footing and would allow direct comparison with published results from cloud reviewers.

15.7 Multi-Turn Repair Loop

The current pipeline is single-shot: the model produces a list of findings and the engine stops. A future version could implement a two-turn loop in which the model is asked to propose a patch, the patch is applied to a temporary copy of the file, and the file is re-reviewed to verify that the patch fixes the issue without introducing a new one. This would deliver real value to users beyond the diagnostic and move the system closer to an autonomous code-fix agent.

15.8 Telemetry, Strictly Opt-In

It would be useful for the development team to know which findings users dismiss, which they act on, and how often the model needs the parser-repair retry. A strictly opt-in, locally-stored telemetry log — never transmitted off the machine — would inform prompt-engineering and analyser-tuning decisions without compromising the privacy property that motivates the system. This must be opt-in and must be local-only; any external transmission would undermine the entire premise of the work.

16. Conclusion

CodeSentinel AI demonstrates that a code reviewer with language-model-level reasoning can run entirely on a personal computer, with no source code leaving the machine and no recurring cost. The reasoning is performed by an open-source model that we run locally; the contribution of this project is the system built around it — the prompt design, the structured-output contract, the JSON response parser, the line-aware chunking and

merging engine, the duplicate-aware fusion with static analysers, the four-format reporting layer, and the dual interface (terminal and graphical). Measured against a four-language benchmark with 38 planted defects, the system found 32 of them — an aggregate recall of 84 % — including every critical security vulnerability in the Python sample. It does so reproducibly, in 10–20 seconds of warm inference per file, within a 6 GB memory budget, and without opening a single non-loopback socket.

The result is a working, documented and tested tool that satisfies the assignment’s constraints exactly: a local model, a graphical interface and a terminal interface, delivered by a team of two. Beyond the assignment, the system stands as a small but concrete answer to a wider question — whether the recent wave of cloud-only AI tools is the only available shape, or whether useful applications of large language models can also be built under strict locality and privacy constraints. Our experience suggests that the second is feasible for a non-trivial class of tasks and is worth pursuing more broadly. The patterns documented here — strict structured output, defensive parsing, tool grounding via classical analysers, and a thin facade over a swappable model — are not specific to code review; they should generalise to any local-LLM application whose output must be consumed downstream by another piece of software.

The work also reinforces a lesson that runs through the literature on applied LLM systems: the model is necessary but not sufficient. The engineering layer around the model — what to ask it, what shape to demand in return, how to recover when the shape is wrong, what to compare its output against, and how to surface the result to the user — is what determines whether the system is a curiosity or a tool. That engineering layer is the substantive contribution of this project, and the part we are most proud of.

References

References are listed in IEEE style, alphabetised by first author.

Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. *In Proceedings of the 35th International Conference on Software Engineering (ICSE 2013)*, pp. 712–721. IEEE.

- Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.
- Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2022). GPT3.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35, pp. 30318–30332.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gerganov, G. (2023). *llama.cpp: Inference of Meta's LLaMA model (and others) in pure C/C++*. GitHub. <https://github.com/ggerganov/llama.cpp>
- Hui, B., Yang, J., Cui, Z., et al. (2024). *Qwen2.5-Coder Technical Report*. arXiv preprint arXiv:2409.12186.
- OASIS Open. (2020). *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. OASIS Standard.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI Technical Report.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, pp. 5998–6008.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *In International Conference on Learning Representations (ICLR 2023)*.
- Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.



Appendix A — Technology Stack

The complete list of third-party components used by CodeSentinel AI is documented in Table A.1. Every component is open source and runs locally; none of them issues a network call as part of normal use.

Component	Choice	Version	Role
Language	Python	3.12	Implementation language of the engine and front ends.
Model runtime	Ollama	0.24	Loads and serves the quantised model locally on 127.0.0.1:11434.
Model	Qwen2.5-Coder	7B-instruct-q4_K_M	Performs the linguistic reasoning over source code.
Underlying engine	llama.cpp	(bundled)	C/C++ inference kernel; Metal on Apple Silicon.
Data modelling	Pydantic	2.8	Typed, validated data structures for Finding and ReviewResult.
Settings	pydantic-settings	2.5	Loads configuration from environment + .env.
Terminal UI	Typer	0.12	Command-line interface with type-driven arg parsing.
Terminal rendering	Rich	13.7	Coloured tables, progress, syntax highlighting.
Graphical UI	Gradio	5.50	Web interface served on localhost; tabbed layout.
Desktop window	pywebview	5.1	Wraps Gradio in a native desktop window.
Templating	Jinja2	3.1	Renders the LLM prompt with the source code.
Static analysis	Bandit	1.7.9	Python security rules; produces JSON output.

Component	Choice	Version	Role
Static analysis	Ruff	0.6	Python style, common bugs and complexity rules.
Static analysis	ESLint	9.x	JavaScript/TypeScript rules; invoked when present.
Fuzzy matching	RapidFuzz	3.9	Duplicate-finding detection in the merger.
HTTP client	httpx	0.27	Async HTTP client to talk to Ollama.
Testing	pytest	8.3	Test runner.
HTTP mocking	respx	0.21	Mocks the httpx calls in unit and functional tests.
Coverage	coverage.py	7.6	Line and branch coverage measurement.

Table A.1 — Complete technology stack.



Appendix B — Reproducing the System

The system is reproducible from the source repository with the following sequence of commands on macOS or Linux. The setup script installs Ollama (if absent), pulls the model, creates the Python virtual environment and installs the project. The full operation takes approximately 10–15 minutes on a typical broadband connection, of which the model download (about 4.7 GB) is the dominant cost.

```
# 1. Clone the repository
git clone https://github.com/<author>/codesentinel-ai.git
cd codesentinel-ai

# 2. Run the bootstrap script (installs Ollama if needed, pulls the model)
bash setup.sh

# 3. Activate the virtual environment
source .venv/bin/activate

# 4. Verify the environment
make doctor

# 5. Run the bundled demo (reviews the sample files)
make demo

# 6. Launch the graphical interface
make web          # opens http://127.0.0.1:7860 in the default browser
# or
codesentinel web --desktop  # opens a native pywebview window

# 7. Try it on your own code
codesentinel review path/to/your/file.py
codesentinel review path/to/project --recursive --fail-on critical
codesentinel diff --repo . --ref HEAD~1
```

The project comprises roughly 3,600 lines of Python across 47 modules, a 55-test suite, a complete Makefile that exposes every operation as a single command, and a README with usage examples. The repository is licensed under MIT and the model itself is governed by its own licence terms (Tongyi Qianwen LICENSE), both of which are reproduced in the repository's LICENSE/ directory.

Source-Code Statistics

Component	Files	Lines of Python
Core engine (codesentinel/)	21	≈ 1900
CLI front end (cli/)	5	≈ 380
Web front end (web/)	6	≈ 520
Static-analyser adapters	4	≈ 280
Tests (tests/)	11	≈ 520
TOTAL	47	≈ 3600

Table B.1 — Source-code statistics.

